

OptiConn: A reliability-screening framework for structural connectomics parameter selection

Karl Koschutnig ¹

¹ MRI-Lab-Graz, University of Graz, Austria

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

Summary

Structural connectome construction depends on numerous interlocking choices (atlas, tracking parameters, streamline count, connectivity threshold, metric). These are often fixed heuristically, reducing reproducibility and risking biased network structure. There is no ground-truth connectome to validate a choice against, so no procedure can claim to find the “correct” or optimal parameter set. *OptiConn* instead provides a transparent, testable screening procedure: it tracks each candidate parameter set multiple times per subject and ranks candidates primarily by **repeat-run discriminability** — how well repeated runs of the same subject are distinguished from other subjects, above tracking noise — after rejecting candidates with implausible graph density or excessive isolated nodes. It supports two search strategies for proposing candidates: a **Bayesian Optimization** engine (Gaussian Processes, for efficient search) and a systematic **Cross-Validation Sweep** (grid/random/Latin hypercube search, for exhaustive baselines). Two tractography backends are supported: **DSI Studio** (default) and, for QSIRecon/QSIPrep users, **MRtrix3** (re-running tckgen/tcksift2/tck2connectome on existing derivatives). The top-ranked, defensible configuration is then applied uniformly to the full cohort, yielding study-specific, reproducible structural connectomes.

Statement of need

There is no consensus “best” parameterization for diffusion tractography-based connectomics; different atlases and thresholds materially shift graph topology, and no ground truth exists to say which topology is “correct”. Existing pipelines (e.g. generic MRtrix or DIPY scripts) provide flexible building blocks but little built-in guidance for principled parameter selection. Exhaustive grid searches are computationally prohibitive (months of runtime), while manual tuning is subjective. *OptiConn* addresses this not by claiming to find an optimal answer, but by making candidate screening explicit and testable: it proposes candidates (via Bayesian or grid/random search), scores them by repeat-run discriminability, and rejects implausible ones — a transparent procedure a reader can inspect and reproduce, rather than a black-box “best” recommendation. *OptiConn* supports both **DSI Studio** and **MRtrix3** (via QSIRecon-preprocessed derivatives) as tractography backends.

State of the field

Foundational graph metrics for brain networks are well established ([Rubinov & Sporns, 2010](#)). Tool ecosystems like MRtrix3, DIPY, and networkx-based wrappers enable tract generation and graph computation but integrate limited automated parameter ranking. Discriminability — the probability that repeated measurements of the same subject are more similar to each other than to another subject’s — has been established as a general criterion for selecting analysis pipelines under exactly this kind of no-ground-truth constraint, including in connectomics ([Bridgeford et al., 2021](#)). *OptiConn* applies that principle specifically to tractography parameter selection: few existing tools implement (1) repeat-run discriminability as the ranking criterion for parameter selection, (2) independent bootstrap-like validation waves, and (3) efficient

candidate proposal via Bayesian search (Snoek et al., 2012) feeding that reliability screen, in a single cohesive, reproducible workflow. *OptiConn* targets that gap.

Design and implementation

Core components:

1. **Bayesian Optimization Engine:** A candidate *proposer* — supporting infrastructure for the discriminability screen in (5), not the selection criterion itself. It uses Gaussian Process regression (via `scikit-optimize`) to model the relationship between tracking parameters (FA, turning angle, step size, etc.) and network quality. It supports **Subject Sampling**, where a different random subject is used for each iteration, ensuring the learned parameters are robust across the population and not overfitted to a single anatomy.
2. **Parameter Sweep Engine:** The engine that applies the discriminability screen in (5). It evaluates candidates — enumerated by grid, random, or Latin hypercube sampling, or handed over from the Bayesian proposer — with repeated tracking runs per subject, under a two-wave cross-validation design that separates optimization from validation.
3. **Computation Integrity Validation:** A safety layer that validates every optimization run. It detects silent failures (e.g., partial connectivity matrices, artificial 1.0 scores) and flags faulty iterations, ensuring that high scores reflect genuine network quality, not artifacts.
4. **Graph Metric Acquisition:** For each candidate, global measures are extracted (density, global efficiency [weighted], small-worldness, clustering/transitivity, path length, assortativity, rich-club indices).
5. **Selection criterion — discriminability:** Candidates are ranked primarily by repeat-run discriminability: the probability that two repeat connectomes from the same subject are more similar to each other than to a connectome from a different subject, above chance. This is computed across real repeat tracking runs and, where available, multiple subjects (`scripts/reliability.py`, `cross_validation_bootstrap_optimizer.py`, `mrtrix_tune.py`), and is the primary criterion driving selection in the sweep engine (`tune-grid`) on both the DSI Studio and MRtrix3 backends, as well as in the MRtrix3 backend's Bayesian mode. The DSI Studio Bayesian sampler proposes candidates by composite quality score; its proposals are screened by discriminability when handed to the sweep engine. A prior composite score, `quality_score_raw`, is still computed and reported alongside it for context — it aggregates five weighted components (Sparsity 25%, Small-Worldness 25%, Modularity 20%, Global Efficiency 20%, Reliability 10%) — but no longer drives selection: three of its five components (small-worldness, modularity, global efficiency) are maximized directly by the score, a known limitation for any metric used as a selection objective, since it rewards configurations for scoring well on the same quantities being optimized rather than for genuine, independently verified topological differences.
6. **Automation & Performance:** Single high-level commands: `tune-bayes` (smart search), `tune-grid` (grid/random search), `select` (promotion), and `apply` (full-cohort extraction). Parallel execution is supported via `--max-workers`.
7. **Reproducibility:** Deterministic seeds, config echoing, parameter snapshots, and machine-readable artifacts (JSON/CSV) ensure every step is traceable.

Quality control

Validation measures:

- **Integrity Checks:** Automatic detection of failed extractions or invalid metrics prevents “garbage in, garbage out”.
- **Automated Test Suite:** A dedicated test suite (`scripts/test_integrity_checks.py`) validates the internal logic of the scoring and failure detection systems, ensuring that edge cases (e.g., NaN values, artificial scores) are handled correctly.

- 92 ▪ **Subject Sampling:** The Bayesian optimizer's ability to sample different subjects per
- 93 iteration prevents overfitting to specific anatomical idiosyncrasies.
- 94 ▪ **Cross-wave Stability:** (Sweep mode) Score divergence between waves flags fragile
- 95 candidates.
- 96 ▪ **Density Corridor:** Rejects extreme sparsity or saturation early.
- 97 ▪ **Diagnostics:** Comprehensive JSON/CSV logs for every iteration allow post-hoc analysis
- 98 of the optimization trajectory.

99 Reuse potential

100 The toolkit enables:

- 101 ▪ **Rapid Screening:** Screen candidate parameters for a new dataset in a few hours using
- 102 Bayesian search plus discriminability scoring.
- 103 ▪ **Method Harmonization:** Share selected_candidate.json files to standardize methods
- 104 across centers.
- 105 ▪ **Benchmarking:** Compare atlases or connectivity metrics under standardized scoring.
- 106 ▪ **Extension:** The modular design allows adding new scorers or optimization strategies.

107 Availability

- 108 ▪ Source repository: [MRI-Lab-Graz/opticonn](#)
- 109 ▪ License: MIT
- 110 ▪ Dependencies: Python 3.10, DSI Studio or MRtrix3 (external, backend-dependent),
- 111 scikit-optimize, numpy/pandas/networkx/scipy.
- 112 ▪ **Demo Data:** To facilitate testing and demonstration, we recommend using the open-
- 113 access diffusion dataset available on OpenNeuro (ds003138) [https://openneuro.org/](https://openneuro.org/datasets/ds003138/versions/1.0.1)
- 114 [datasets/ds003138/versions/1.0.1](https://openneuro.org/datasets/ds003138/versions/1.0.1). This dataset is compatible with the BIDS standard
- 115 and suitable for testing the pipeline's capabilities.

116 Acknowledgements

117 We acknowledge the support of the MRI-Lab-Graz team for testing and feedback.

118 References

- 119 Bridgeford, E. W., Wang, S., Wang, Z., Xu, T., Craddock, C., Dey, J., Kiar, G., Gray-Roncal,
- 120 W., Colantuoni, C., Douville, C., Noble, S., Priebe, C. E., Caffo, B., Milham, M., Zuo, X.-N.,
- 121 & Vogelstein, J. T. (2021). Eliminating accidental deviations to minimize generalization
- 122 error and maximize replicability: Applications in connectomics and genomics. *PLOS*
- 123 *Computational Biology*, 17(9), e1009279. <https://doi.org/10.1371/journal.pcbi.1009279>
- 124 Rubinov, M., & Sporns, O. (2010). Complex network measures of brain connectivity: Uses and
- 125 interpretations. *NeuroImage*, 52(3), 1059–1069. [https://doi.org/10.1016/j.neuroimage.](https://doi.org/10.1016/j.neuroimage.2009.10.003)
- 126 [2009.10.003](https://doi.org/10.1016/j.neuroimage.2009.10.003)
- 127 Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical bayesian optimization of machine
- 128 learning algorithms. *Advances in Neural Information Processing Systems*, 25.